

TP n°3 : Gestion d'une Médiathèque

Ressource R3.04 – Qualité de Développement • Documentation & Synthèse Pédagogique

Module : R3.04 Qualité de développement (Java)

Enseignant : Sébastien Thon

Niveau : BUT Informatique – Semestre 3

Conformité : 100% basé sur le cours officiel (Chapitres 1 à 10) et TP1 / TP2

1. Introduction & Objectifs Pédagogiques

Le TP n°3 met en application les principes clés de la programmation orientée objet en Java vus en cours magistral. À travers la réalisation d'une application en console de gestion d'une médiathèque hébergeant des livres et des DVD, ce travail valide :

- **L'encapsulation et le contrôle des visibilitées** : distinction stricte entre variables privées (`private`) et protégées (`protected`) selon les spécifications.
- **L'héritage simple** : dérivation de classes spécialisées (`Livre`, `DVD`) à partir d'une classe de base générique (`Ouvrage`) avec le mot-clé `extends`.
- **La réutilisation de constructeurs** : délégation de l'initialisation à la classe mère via l'instruction `super(...)`.
- **La redéfinition de méthodes (Overriding)** : spécialisation de l'affichage avec l'annotation `@Override` et appel de la méthode parente via `super.afficher()`.
- **Les collections dynamiques génériques** : utilisation de la classe `ArrayList<Ouvrage>` pour dimensionner la collection dynamiquement.
- **Le polymorphisme d'exécution** : traitement unifié des ouvrages au sein de la médiathèque permettant d'invoquer la méthode d'affichage adéquate selon la nature réelle de l'objet.
- **L'interactivité console** : boucle d'exécution avec menu interactif `do ... while / switch` et gestion fiable des flux de saisie via `Scanner`.

2. Tableau de Concordance : Syntaxe Utilisée vs Références du Cours

Conformément aux exigences de rigueur, **absolument aucune syntaxe externe ou non enseignée n'a été employée**. Chaque construction syntaxique correspond rigoureusement aux diapositives du cours de Sébastien Thon :

Construction Java	Rôle dans le TP03	Référence exacte Cours R3.04
<code>private</code> / <code>protected</code> / <code>public</code>	Encapsulation de <code>numero</code> (privé), <code>emprunte</code> (protégé), et données des classes dérivées	Chapitre 3 – Diapo 20 « Les 3 niveaux de protection des données : <code>public</code> , <code>private</code> , <code>protected</code> »
<code>class B extends A</code>	Héritage de <code>Livre</code> et <code>DVD</code> depuis <code>Ouvrage</code>	Chapitre 3 – Diapo 22 « Héritage : Syntaxe <code>Class B extends A</code> »
Constructeur & <code>this</code>	Initialisation des champs dans les constructeurs	Chapitre 3 – Diapos 10 & 13 « Constructeur d'une classe et mot-clé <code>this</code> »
<code>super(...)</code> (constructeur)	Appel du constructeur d' <code>Ouvrage</code> depuis <code>Livre</code> et <code>DVD</code>	Chapitre 3 – Diapos 25 & 27 « Appel direct d'un constructeur de la classe mère avec <code>super</code> »
<code>@Override</code>	Redéfinition de <code>afficher()</code> dans <code>Livre</code> et <code>DVD</code>	Chapitre 3 – Diapos 28 & 29 « Annotation <code>@Override</code> : vérification de l'authenticité de la surcharge »

Construction Java	Rôle dans le TP03	Référence exacte Cours R3.04
<code>super.afficher()</code>	Affichage des attributs communs d'Ouvrage dans les sous-classes	Chapitre 3 – Diapo 25 « Appel d'une méthode de la super-classe avec super »
<code>ArrayList<Ouvrage></code>	Structure de stockage dynamique des ouvrages	Chapitre 6 – Diapo 19 « 3. Le type ArrayList (tableaux redimensionnables génériques) »
<code>liste.add(elem)</code>	Ajout d'un livre ou DVD à la fin de la collection	Chapitre 6 – Diapo 22 « 1) Ajout d'élément à une ArrayList avec add() »
<code>liste.get(i)</code>	Lecture d'un ouvrage par son indice dans la boucle	Chapitre 6 – Diapo 23 « 2) Lire un élément d'une ArrayList avec get() »
<code>size()</code> & test <code>size() == 0</code>	Vérification de liste vide et borne de boucle de parcours	Chapitre 6 – Diapos 26 & 43 « Connaître le nombre d'éléments avec size() »
Polymorphisme dynamique	<code>listeOuvrages.get(i).afficher()</code> résolu selon le type réel	Chapitre 7 – Diapo 14 & Chapitre 3 – Diapo 25 « Polymorphisme : écrire du code générique sur les types dérivés »
<code>Scanner (nextInt, nextByte, nextLine)</code>	Saisies des données et purge du retour à la ligne	Chapitre 2 – Diapos 14 & 15 « Saisie de données au clavier avec Scanner et purge avec nextLine() »
<code>equalsIgnoreCase(...)</code>	Comparaison de la réponse d'emprunt ("oui")	Chapitre 4 – Diapo 10 « boolean equalsIgnoreCase(autre_mot) sans différencier la casse »
Menu <code>do ... while / switch</code>	Structure globale de la boucle du programme principal	Chapitre 6 – Diapos 14 & 15 « Exemple d'application interactive : GestionClasse.java »

3. Architecture Globale de la Solution

L'application s'articule autour de 5 fichiers sources situés dans le dossier `TP03/`, respectant le principe de responsabilité unique :

Diagramme de Classes et Relations (UML)

```
+-----+
|               Ouvrage               |
+-----+
| - numero: int                       |
| # emprunte: boolean                 |
+-----+
| + Ouvrage(numero: int, emprunte: boolean)|
| + getNumero(): int                  |
| + setNumero(numero: int): void      |
| + getEmprunte(): boolean            |
| + isEmprunte(): boolean             |
| + setEmprunte(emprunte: boolean): void
| + afficher(): void                  |
+-----+

      ^
      | extends
      +-----+
      |               |               |
+-----+             +-----+
|               Livre               | |               DVD               |
+-----+             +-----+
| - titre: String                   | | - titre: String                   |
| - auteur: String                  | | - duree: int                      |
+-----+             +-----+
| + Livre(num, emp, titre, auteur)  | | + DVD(num, emp, titre, duree)    |
| + getTitre(): String              | | + getTitre(): String              |
| + setTitre(titre: String): void   | | + setTitre(titre: String): void   |
| + getAuteur(): String             | | + getDuree(): int                  |
| + setAuteur(auteur: String): void | | + setDuree(duree: int): void      |
| + afficher(): void [@Override]    | | + afficher(): void [@Override]    |
+-----+             +-----+

+-----+             +-----+
|               Mediatheque             | |               Principal             |
+-----+             +-----+
| - listeOuvrages: ArrayList<Ouvrage> | |<-----| + main(args): void|
+-----+             +-----+
| + Mediatheque()                    | |
| + getListeOuvrages(): ArrayList<Ouvrage>
| + ajouterOuvrage(): void            | |
| + ajouterOuvrage(clavier: Scanner): void
| + afficherOuvrages(): void          | |
+-----+             +-----+
```

4. Analyse Détaillée Classe par Classe et Fonction par Fonction

4.1 Classe `Ouvrage.java`

Justification Cours R3.04 : Chapitre 3 (Diapo 20 pour la visibilité `private` et `protected`, Diapo 13 pour le constructeur, Diapo 21 pour les accesseurs).

La classe `Ouvrage` est la classe mère générale. Elle stocke les deux propriétés communes :

- `private int numero` : identifiant d'ouvrage. Il est déclaré privé, interdisant tout accès direct extérieur ou hérité (les accès se font via `getNumero()` et `setNumero()`).
- `protected boolean emprunte` : statut d'emprunt. Il est déclaré protégé, permettant aux classes dérivées (`Livre`, `DVD`) d'y accéder directement conformément aux spécifications du sujet.
- `public void afficher()` : affiche le numéro et le statut de disponibilité.

```
public class Ouvrage {
    private int numero;
    protected boolean emprunte;

    // Constructeur initialisant les attributs (Chapitre 3, Diapo 13)
    public Ouvrage(int numero, boolean emprunte) {
        this.numero = numero;
        this.emprunte = emprunte;
    }

    // Accesseurs et mutateurs (Chapitre 3, Diapo 21)
    public int getNumero() { return this.numero; }
    public void setNumero(int numero) { this.numero = numero; }

    public boolean getEmprunte() { return this.emprunte; }
    public boolean isEmprunte() { return this.emprunte; }
    public void setEmprunte(boolean emprunte) { this.emprunte = emprunte; }

    // Méthode d'affichage des caractéristiques d'un ouvrage
    public void afficher() {
        System.out.println("Numéro d'identification : " + this.numero);
        if (this.emprunte) {
            System.out.println("Statut : Emprunté");
        } else {
            System.out.println("Statut : Disponible");
        }
    }
}
```

4.2 Classe Livre.java

Justification Cours R3.04 : Chapitre 3 (Diapo 22 pour `extends`, Diapos 25 & 27 pour `super(...)`, Diapos 28 & 29 pour `@Override`, Diapo 25 pour `super.afficher()`).

La classe `Livre` spécialise `Ouvrage` en lui ajoutant un `titre` et un `auteur` :

- `super(numero, emprunte)` : invoque le constructeur de la classe de base pour lui transmettre les paramètres d'initialisation du numéro et du statut.
- `@Override public void afficher()` : redéfinit la méthode d'affichage en appelant préalablement `super.afficher()` pour mutualiser l'affichage commun, puis affiche les informations spécifiques du livre.

```
public class Livre extends Ouvrage {
    private String titre;
    private String auteur;

    // Constructeur recevant données spécifiques + données de base (Chapitre 3, Diapos 25-27)
    public Livre(int numero, boolean emprunte, String titre, String auteur) {
        super(numero, emprunte); // Délégation à la super-classe
        this.titre = titre;
        this.auteur = auteur;
    }

    public String getTitre() { return this.titre; }
    public void setTitre(String titre) { this.titre = titre; }

    public String getAuteur() { return this.auteur; }
    public void setAuteur(String auteur) { this.auteur = auteur; }

    @Override // Chapitre 3, Diapos 28-29
    public void afficher() {
        System.out.println("--- Livre ---");
        super.afficher(); // Chapitre 3, Diapo 25 : réutilisation méthode classe mère
        System.out.println("Titre : " + this.titre);
        System.out.println("Auteur : " + this.auteur);
    }
}
```

4.3 Classe DVD.java

Justification Cours R3.04 : Même principe de dérivation et de réutilisation avec `super` qu'au Chapitre 3.

La classe `DVD` spécialise également `Ouvrage` en encapsulant le `titre` et la `duree` en minutes :

```
public class DVD extends Ouvrage {
    private String titre;
    private int duree;

    public DVD(int numero, boolean emprunte, String titre, int duree) {
        super(numero, emprunte); // Chapitre 3, Diapos 25 et 27
        this.titre = titre;
        this.duree = duree;
    }

    public String getTitre() { return this.titre; }
    public void setTitre(String titre) { this.titre = titre; }

    public int getDuree() { return this.duree; }
    public void setDuree(int duree) { this.duree = duree; }

    @Override
    public void afficher() {
        System.out.println("--- DVD ---");
        super.afficher(); // Appel caractéristiques Ouvrage
        System.out.println("Titre : " + this.titre);
        System.out.println("Durée : " + this.duree + " minutes");
    }
}
```

4.4 Classe Mediatheque.java

Justification Cours R3.04 : Chapitre 6 (Diapo 19 pour `ArrayList<Ouvrage>`, Diapo 22 pour `add()`, Diapo 23 pour `get()`, Diapo 26 & 43 pour `size() == 0`); Chapitre 4 (Diapo 10 pour `equalsIgnoreCase()`); Chapitre 2 (Diapos 14-15 pour `Scanner`).

Cette classe gère l'inventaire des documents à l'aide d'une collection `ArrayList<Ouvrage>` :

- `ajouterOuvrage(Scanner clavier)` : aiguille la création vers un `Livre` ou un `DVD`, lit les données au clavier avec purge du retour chariot (`clavier.nextLine()`, Chapitre 2 diapo 15), instancie l'objet et l'ajoute via `this.listeOuvrages.add(...)`.
- `afficherOuvrages()` : teste la présence d'ouvrages avec `size() == 0` (Chapitre 6 diapo 26 & 43), puis boucle sur la liste pour invoquer `this.listeOuvrages.get(i).afficher()`. Le **polymorphisme** permet à la JVM d'exécuter la méthode appropriée (`Livre` ou `DVD`) de façon transparente.

```
import java.util.ArrayList; // Chapitre 6, Diapo 19
import java.util.Scanner;   // Chapitre 2, Diapo 14

public class Mediatheque {
    private ArrayList<Ouvrage> listeOuvrages;

    public Mediatheque() {
        this.listeOuvrages = new ArrayList<Ouvrage>();
    }

    public ArrayList<Ouvrage> getListeOuvrages() {
        return this.listeOuvrages;
    }

    public void ajouterOuvrage() {
        Scanner clavier = new Scanner(System.in);
        ajouterOuvrage(clavier);
    }

    public void ajouterOuvrage(Scanner clavier) {
        System.out.println("
        Quel type d'ouvrage voulez-vous ajouter ?");
        System.out.println("1. Livre");
        System.out.println("2. DVD");
        System.out.print("Votre choix : ");
        int type = clavier.nextInt();
        clavier.nextLine(); // Purge retour ligne (Chapitre 2, Diapo 15)

        if (type != 1 && type != 2) {
            System.out.println("Type d'ouvrage invalide.");
            return;
        }

        System.out.print("Numéro d'identification : ");
        int numero = clavier.nextInt();
        clavier.nextLine();

        System.out.print("L'ouvrage est-il emprunté ? (oui/non) : ");
        String repEmprunt = clavier.nextLine();
        // equalsIgnoreCase vu au Chapitre 4, Diapo 10
        boolean emprunte = repEmprunt.equalsIgnoreCase("oui");

        if (type == 1) {
            System.out.print("Titre du livre : ");
            String titre = clavier.nextLine();
            System.out.print("Nom de l'auteur : ");
            String auteur = clavier.nextLine();

            Livre livre = new Livre(numero, emprunte, titre, auteur);
            this.listeOuvrages.add(livre); // Chapitre 6, Diapo 22
            System.out.println("Livre ajouté avec succès !");
        } else {
            System.out.print("Titre du DVD : ");
            String titre = clavier.nextLine();
        }
    }
}
```

```

        System.out.print("Durée du DVD (en minutes) : ");
        int duree = clavier.nextInt();
        clavier.nextLine();

        DVD dvd = new DVD(numero, emprunte, titre, duree);
        this.listeOuvrages.add(dvd); // Chapitre 6, Diapo 22
        System.out.println("DVD ajouté avec succès !");
    }
}

public void afficherOuvrages() {
    // Test de liste vide via size() (Chapitre 6, Diapos 26 & 43)
    if (this.listeOuvrages.size() == 0) {
        System.out.println("
Aucun ouvrage dans la médiathèque.");
        return;
    }

    System.out.println("
==== Liste des ouvrages de la médiathèque (" + this.listeOuvrages.size() + ") ====");
    // Parcours par indice (Chapitre 6, Diapo 23 & Diapos 50-51)
    for (int i = 0; i < this.listeOuvrages.size(); i++) {
        // Résolution dynamique polymorphique (Chapitre 7, Diapo 14)
        this.listeOuvrages.get(i).afficher();
        System.out.println("-----");
    }
}
}

```

4.5 Classe Principal.java

Justification Cours R3.04 : Chapitre 6 (Diapos 14 et 15, fichier modèle GestionClasse.java).

Point d'entrée contenant la méthode `main`. Elle applique le canevas type du cours : instantiation de l'objet gestionnaire, lecture du choix et dispatching avec `switch` (choix) dans une boucle `do ... while` (choix != 3) :

```
import java.util.Scanner;

public class Principal {
    public static void main(String[] args) {
        Mediatheque mediatheque = new Mediatheque();
        Scanner clavier = new Scanner(System.in);
        byte choix = 0;

        // Boucle de menu calquée sur GestionClasse.java (Chapitre 6, Diapos 14-15)
        do {
            System.out.println("
=== Menu Médiathèque ===");
            System.out.println("1. Ajouter un ouvrage");
            System.out.println("2. Afficher la liste des ouvrages");
            System.out.println("3. Quitter");
            System.out.print("Votre choix : ");

            choix = clavier.nextByte();
            clavier.nextLine(); // Consomme le retour à la ligne

            switch (choix) {
                case 1:
                    mediatheque.ajouterOuvrage(clavier);
                    break;
                case 2:
                    mediatheque.afficherOuvrages();
                    break;
                case 3:
                    System.out.println("Au revoir !");
                    break;
                default:
                    System.out.println("Option inexistante.");
            }
        } while (choix != 3);
    }
}
```

5. Validation et Déroulement des Tests

Le projet compile sans aucune anomalie et a été exécuté sur un scénario complet d'ajout et de consultation :

Trace d'exécution en console

```
=== Menu Médiathèque ===
1. Ajouter un ouvrage
2. Afficher la liste des ouvrages
3. Quitter
Votre choix : 1

Quel type d'ouvrage voulez-vous ajouter ?
1. Livre
2. DVD
Votre choix : 1
Numéro d'identification : 101
L'ouvrage est-il emprunté ? (oui/non) : non
Titre du livre : Le Petit Prince
Nom de l'auteur : Antoine de Saint-Exupéry
Livre ajouté avec succès !

=== Menu Médiathèque ===
1. Ajouter un ouvrage
2. Afficher la liste des ouvrages
3. Quitter
Votre choix : 1

Quel type d'ouvrage voulez-vous ajouter ?
1. Livre
2. DVD
Votre choix : 2
Numéro d'identification : 202
L'ouvrage est-il emprunté ? (oui/non) : oui
Titre du DVD : Inception
Durée du DVD (en minutes) : 148
DVD ajouté avec succès !

=== Menu Médiathèque ===
1. Ajouter un ouvrage
2. Afficher la liste des ouvrages
3. Quitter
Votre choix : 2

===== Liste des ouvrages de la médiathèque (2) =====
--- Livre ---
Numéro d'identification : 101
Statut : Disponible
Titre : Le Petit Prince
Auteur : Antoine de Saint-Exupéry
-----
--- DVD ---
Numéro d'identification : 202
Statut : Emprunté
Titre : Inception
Durée : 148 minutes
-----

=== Menu Médiathèque ===
1. Ajouter un ouvrage
2. Afficher la liste des ouvrages
3. Quitter
Votre choix : 3
Au revoir !
```

6. Bilan Pédagogique, Qualité de Code & Perspectives

6.1 Bilan Technique et Notions Clés Validées

Ce TP n°3 constitue une étape pivot du module R3.04. Il consolide l'ensemble des mécanismes fondamentaux de la programmation orientée objet en Java :

- **Encapsulation et modularité** : Respect rigoureux des portées `private` et `protected` conformément aux spécifications du sujet. L'état interne de chaque classe est protégé des altérations directes externes, tout en permettant aux classes dérivées d'interagir efficacement avec la classe de base.
- **Héritage et réutilisabilité** : Mutualisation de la logique commune dans la classe mère `Ouvrage` et spécialisation sans redondance au sein des classes filles `Livre` et `DVD` grâce aux appels systématiques à `super(...)` et `super.afficher()`.
- **Polymorphisme à l'exécution** : Utilisation d'une collection hétérogène `ArrayList<Ouvrage>` capable de stocker indifféremment des instances de `Livre` et de `DVD`, tout en déléguant à la machine virtuelle Java la résolution dynamique des méthodes d'affichage spécifiques lors du parcours.
- **Stricte orthodoxie du cours** : Utilisation exclusive des idiomes enseignés en cours magistral (`Scanner` avec purge explicite de ligne, boucle interactive `do ... while / switch` calquée sur `GestionClasse.java`).

6.2 Tableau Récapitulatif des Bonnes Pratiques Appliquées

Critère Qualité R3.04	Mise en œuvre dans le TP03	Bénéfice Développement
Conventions de nommage Java	Majuscule pour les classes (<code>Ouvrage</code> , <code>Mediatheque</code>), camelCase pour méthodes/variables.	Lisibilité et homogénéité du code selon les normes professionnelles Java.
Délégation au constructeur parent	<code>super(numero, emprunte)</code> en première ligne des constructeurs dérivés.	Garantit l'initialisation ordonnée et robuste de l'ensemble de l'état objet.
Annotation <code>@Override</code>	Placée devant chaque méthode <code>afficher()</code> redéfinie dans <code>Livre</code> et <code>DVD</code> .	Vérification à la compilation contre les fautes de frappe de signature (Chapitre 3, Diapo 29).
Gestion propre des flux I/O	Purge systématique du retour chariot par <code>clavier.nextLine()</code> après lecture d'entier.	Évite tout saut inopiné de saisie texte lors des entrées interactives en console.

6.3 Perspectives et Évolutions Naturelles (Chapitres Suivants)

L'architecture ainsi posée offre une évolutivité naturelle vers les compétences abordées dans les chapitres ultérieurs du cours R3.04 :

- **Archivage et persistance (Chapitre 9)** : Possibilité de sauvegarder et charger automatiquement la liste des ouvrages sur disque au moyen de flux de données (fichiers texte ou sérialisation d'objets via `ObjectOutputStream`).
- **Gestion robuste des exceptions (Chapitre 10)** : Encadrement des saisies utilisateur par des blocs `try { ... } catch (InputMismatchException e) { ... }` afin d'empêcher toute interruption du programme en cas de saisie invalide.
- **Abstraction et contrats (Chapitre 7)** : Évolution éventuelle d'`Ouvrage` en classe abstraite (`abstract class ouvrage`) ou implémentation d'interfaces métier (ex: `Empruntable`).